

Painel de Projetos

Case Técnico — PVT Software & Serviços

Desenvolvedor: Pedro Henrique Souza

Vaga: Desenvolvedor Web e Infraestrutura JR

Stack: React + TypeScript · Node.js · PostgreSQL

Junho de

2026

Sumário

1	Modelagem do Domínio	2
1.1	Entidades e Atributos	2
1.2	Relacionamentos	3
2	Arquitetura da Solução	4
2.1	Camadas	4
2.2	Decisão Central: Sincronização Assíncrona com o ERP	4
3	Design da API	5
3.1	Resumo dos Endpoints	5
3.2	Exemplos de Requisição e Resposta	5
4	Regras de Negócio	7
4.1	Saldo de Horas	7
4.2	Percentual de Avanço	7
4.3	Classificação de Status	7
5	Decisões e Trade-offs	9
5.1	PostgreSQL em vez de MongoDB	9
5.2	Sincronização Assíncrona em vez de Consulta Direta ao ERP	9
5.3	Cálculos na API, não no Frontend	9
5.4	Arquitetura em Camadas em vez de Monólito Simples	9
5.5	Threshold de Status Configurável	9
5.6	Endpoint de Dashboard Separado	9
6	Como eu evoluiria isso	10
6.1	Performance e Escala	10
6.2	Resiliência a Falhas	10

Modelagem do Domínio

O módulo Painel de Projetos é construído sobre cinco entidades principais que representam os conceitos do negócio: o trabalho contratado, quem contratou, quem executa, como os profissionais são vinculados aos projetos e como as horas são registradas.

Entidades e Atributos

Cliente

Representa a empresa que contratou a PVT.

Campo	Tipo	Descrição
id	INT	Chave primária
nome	STRING	Nome da empresa
email	STRING	E-mail de contato

Projeto

O trabalho contratado pelo cliente. Guarda horas vendidas (contratuais) e horas planejadas (estimativa interna da equipe).

Campo	Tipo	Descrição
id	INT	Chave primária
nome	STRING	Nome do projeto
dataInicio	DATE	Data de início
dataFim	DATE	Prazo de entrega
horasVendidas	DECIMAL	Total de horas contratadas
horasPlanejadas	DECIMAL	Estimativa interna de horas
id_cliente (FK)	INT	Referência ao Cliente

Analista

Profissional alocado nos projetos.

Campo	Tipo	Descrição
id	INT	Chave primária
nome	STRING	Nome completo
email	STRING	E-mail único

Alocação

Entidade associativa que conecta Analista e Projeto. Permite que um analista esteja em múltiplos projetos simultaneamente e que um projeto tenha múltiplos analistas.

Campo	Tipo	Descrição
id	INT	Chave primária
id_projeto (FK)	INT	Referência ao Projeto
id_analista (FK)	INT	Referência ao Analista

Apontamento

Registro diário de horas trabalhadas por um analista dentro de uma alocação.

Campo	Tipo	Descrição
id	INT	Chave primária
id_alocacao (FK)	INT	Referência à Alocação
data	DATE	Data do trabalho realizado
horas	DECIMAL	Quantidade de horas apontadas

Relacionamentos

- Um **Cliente** possui vários **Projetos** (1:N)
- Um **Projeto** tem várias **Alocações** (1:N)
- Um **Analista** pode ter várias **Alocações** em projetos diferentes (1:N)
- Cada **Alocação** gera vários **Apontamentos** de horas (1:N)

Arquitetura da Solução

A solução é organizada em quatro camadas com responsabilidades distintas, seguindo o princípio de separação de responsabilidades.

Camadas

1. **Frontend** — Interface React + TypeScript acessada pelo coordenador no navegador
2. **API / Backend** — Servidor Node.js responsável pelas regras de negócio, cálculos e autenticação
3. **Banco de Dados** — PostgreSQL armazenando projetos, alocações e apontamentos
4. **Serviço de Sincronização** — Job agendado que busca dados do ERP TOTVS RM periodicamente

Decisão Central: Sincronização Assíncrona com o ERP

A API **nunca consulta o ERP diretamente** em tempo de requisição. Um serviço separado busca os dados do TOTVS RM de hora em hora e atualiza o banco local. A API responde sempre a partir do banco.

Por que sincronização assíncrona?

- O painel continua funcionando mesmo quando o ERP está fora do ar
- O tempo de resposta da API não depende da disponibilidade do ERP
- Dado com no máximo 1h de defasagem — aceitável para gestão de projetos

Design da API

O módulo Painel de Projetos é disponibilizado através de uma API REST. Os principais recursos são projetos, analistas, alocações e apontamentos.

Resumo dos Endpoints

Verbo + Endpoint	Descrição
GET /api/projetos	Lista todos os projetos com horas vendidas, realizadas, saldo e status
GET /api/projetos/{id}	Retorna os detalhes de um projeto específico
POST /api/projetos	Cria um novo projeto
GET /api/analistas	Lista todos os analistas cadastrados
POST /api/alocacoes	Aloca um analista em um projeto
POST /api/apontamentos	Registra horas trabalhadas por um analista
GET /api/projetos/{id}/apontamentos	Lista apontamentos de um projeto
GET /api/projetos/{id}/dashboard	Retorna indicadores gerenciais do projeto
POST /api/integracao/rm/sincronizar	Dispara sincronização com o ERP TOTVS RM

Exemplos de Requisição e Resposta

GET /api/projetos

```
1 [
2   {
3     "id": 1,
4     "nome": "Implantacao ERP",
5     "horasVendidas": 200,
6     "horasRealizadas": 150,
7     "saldoHoras": 50,
8     "status": "SAUDAVEL"
9   }
10 ]
```

Listing 1: Exemplo de resposta

GET /api/projetos/{id}/dashboard

```
1 {  
2   "horasVendidas": 200,  
3   "horasPlanejadas": 180,  
4   "horasRealizadas": 150,  
5   "saldoHoras": 50,  
6   "percentualAvanco": 75,  
7   "status": "SAUDAVEL"  
8 }
```

Listing 2: Indicadores gerenciais do projeto

POST /api/apontamentos

```
1 // Requisicao  
2 {  
3   "idAlocacao": 1,  
4   "data": "2026-06-25",  
5   "horas": 8  
6 }  
7  
8 // Resposta  
9 {  
10  "mensagem": "Apontamento registrado com sucesso"  
11 }
```

Listing 3: Registro de apontamento de horas

Regras de Negócio

Saldo de Horas

O saldo representa quantas horas ainda restam no contrato. É calculado subtraindo as horas já realizadas das horas vendidas ao cliente.

$$\text{saldoHoras} = \text{horasVendidas} - \text{horasRealizadas}$$

$$\text{horasRealizadas} = \sum_{\text{apontamentos do projeto}} \text{horas}$$

Resultado	Interpretação
Positivo	Horas disponíveis no contrato
Zero	Horas esgotadas, mas não estouradas
Negativo	Projeto estourou as horas contratadas

Percentual de Avanço

Indica quanto do orçamento de horas já foi consumido. Deve ser comparado com o avanço físico das entregas para detectar descompassos.

$$\text{percentualAvanco} = \frac{\text{horasRealizadas}}{\text{horasVendidas}} \times 100$$

Exemplo: se o projeto está em 80% das horas mas apenas 50% das entregas foram concluídas, há um problema de ritmo que exige atenção da coordenação.

Classificação de Status

O status combina duas dimensões: consumo de horas e prazo de entrega.

saldo > 10% das horasVendidas E prazo não vencido
Projeto dentro do esperado — horas sobrando e entrega no prazo.

0 < saldo ≤ 10% das horasVendidas OU prazo em até 7 dias
Horas acabando ou data de entrega se aproximando. Requer acompanhamento.

saldo < 0 (horas estouradas) OU prazo vencido
Projeto consumiu mais horas do que o contratado ou a entrega está atrasada.

Implementação em Node.js

```
1 function calcularStatus(horasVendidas, horasRealizadas,
2   dataFimPrevista) {
3   const saldo = horasVendidas - horasRealizadas;
4   const hoje = new Date();
5   const dataFim = new Date(dataFimPrevista);
6   const diasRestantes = Math.ceil(
7     (dataFim - hoje) / (1000 * 60 * 60 * 24)
8   );
9   const limiteAtencao = horasVendidas * 0.10;
10
11   if (saldo < 0 || diasRestantes < 0) return "CRITICO";
12   if (saldo <= limiteAtencao || diasRestantes <= 7) return "
13     ATENCAO";
14   return "SAUDAVEL";
15 }
```

Listing 4: Função de cálculo de status

Nota: O threshold de 10% é configurável via parâmetro no banco de dados, permitindo que a coordenação ajuste o valor sem necessidade de novo deploy.

Decisões e Trade-offs

PostgreSQL em vez de MongoDB

Os dados do domínio são fortemente relacionais — projetos têm clientes, alocações conectam analistas a projetos, apontamentos pertencem a alocações. Um banco relacional garante integridade referencial via *foreign keys* e facilita queries de agregação (SUM de horas por projeto). MongoDB não teria justificativa técnica nesse contexto.

Sincronização Assíncrona em vez de Consulta Direta ao ERP

Consultar o ERP diretamente a cada requisição criaria dois problemas: dependência de disponibilidade (se o ERP cair, o painel para) e latência (cada clique dependeria do tempo de resposta do TOTVS RM). A sincronização periódica desacopla os dois sistemas, com custo de até 1h de defasagem nos dados — aceitável para gestão de projetos.

Cálculos na API, não no Frontend

Centralizar saldo, percentual e status na API garante que qualquer cliente (web, mobile, relatórios) receba o mesmo resultado sem reimplementar lógica. Se a regra de "CRÍTICO" mudar, a alteração acontece em um único lugar, sem necessidade de atualizar múltiplos clientes.

Arquitetura em Camadas em vez de Monólito Simples

A separação em camadas segue o princípio de responsabilidade única. Cada camada pode evoluir, ser testada e escalada de forma independente. Um monólito seria mais simples no início, mas difícil de manter conforme o portal cresce com novos módulos.

Threshold de Status Configurável

O limite de 10% para o status “ATENCAO” é uma decisão de negócio, não técnica. Torná-lo configurável (parâmetro salvo no banco) permite que a coordenação ajuste sem envolver desenvolvimento — o sistema serve à empresa, não o contrário.

Endpoint de Dashboard Separado

O endpoint `GET /api/projetos/{id}/dashboard` foi criado como recurso dedicado para indicadores gerenciais, separando a responsabilidade de “listar projetos” de “exibir métricas detalhadas”. Isso permite que o frontend carregue a listagem rapidamente e só busque o dashboard quando o usuário selecionar um projeto específico.

Como eu evoluiria isso

Performance e Escala

Cache nos endpoints mais acessados

O `GET /api/projetos` será chamado toda vez que alguém abrir o painel. Com centenas de projetos e vários acessos simultâneos, cada requisição executa agregações pesadas no banco. Adicionar cache (ex: Redis) com TTL de 5 minutos faz a API responder sem bater no banco a cada requisição. O cache é invalidado automaticamente quando um novo apontamento é registrado.

Índices no banco de dados

Criar índices em `apontamentos.id_alocacao` e `alocacoes.id_projeto` acelera significativamente as queries de agregação de horas. Sem índices, o banco percorre a tabela inteira a cada soma — inaceitável em escala.

Paginação nos endpoints de listagem

Em vez de retornar todos os projetos de uma vez, adotar `GET /api/projetos?page=1&limit=20`. O frontend carrega sob demanda, reduzindo o volume de dados trafegados e o tempo de resposta percebido pelo usuário.

Resiliência a Falhas

ERP fora do ar

Como a API nunca consulta o ERP em tempo real, uma queda do TOTVS RM não afeta o painel. O serviço de sync registra a falha em log e tenta novamente na próxima janela agendada. A interface exibe o timestamp da última sincronização bem-sucedida para transparência com o usuário.

Circuit Breaker no serviço de sincronização

Se o ERP estiver lento (não fora do ar, apenas com alta latência), o serviço de sync pode ficar travado aguardando resposta. Um *circuit breaker* detecta N tentativas sem sucesso e interrompe as tentativas por um período determinado, evitando que o serviço consuma recursos desnecessariamente.

Logs, monitoramento e alertas

Em produção, registrar logs de cada execução do job de sync (sucesso, falha, tempo de resposta, registros atualizados) e monitorar o tempo de resposta da API. Alertas automáticos devem ser disparados se a sync falhar consecutivamente ou se a API apresentar degradação de performance — antes que o usuário perceba.